

# 后台异步任务迁移方案

## 目标

把 EduAgent 从当前的“API 进程内同步执行任务”改造成真正的后台任务架构，让耗时任务脱离 FastAPI 请求生命周期，在独立 worker 进程中运行。

适合迁移到后台 worker 的任务包括：

- 文档解析、分块、embedding、索引入库
- flashcard 生成
- quiz 生成
- note 生成
- mind map 生成
- 后续的 chat title 生成

聊天回复的流式输出应继续留在 API 进程内，因为它和当前 HTTP/SSE 响应直接绑定。

## 当前状态

当前的 QueueService 不是真正的队列。它会在 API 进程里立刻调用任务处理函数：

- `src/shared/queue/src/edu_queue/service.py`
- `src/edu-api/task_runner.py`
- `src/edu-api/dependencies.py`

当前调用链：

```
API endpoint 或 chat tool
-> service.queue_generation(...)
-> QueueService.send_message(...)
-> TaskRunnerService.dispatch(...)
-> TaskRunnerService._dispatch_async(...)
-> generation/document task 在 API 进程内执行
```

这意味着项目里虽然有 `async def`，但长任务并不持久，也没有真正交给独立 worker 处理。

# 主流方案调研

2026 年常见选择：

| 方案                           | 适配度       | 说明                                                                    |
|------------------------------|-----------|-----------------------------------------------------------------------|
| FastAPI<br>BackgroundTasks   | 不适合本项目    | 适合响应后做轻量工作；FastAPI 文档也建议重任务使用 Celery 这类更大的工具。                         |
| Celery +<br>Redis/RabbitMQ   | 生产环境强默认选项 | 最成熟的分布式任务队列，生态完整，支持重试、路由、监控。缺点是和 async-first 代码结合时需要额外包装。             |
| Dramatiq +<br>Redis/RabbitMQ | 可选的简化方案   | 比 Celery 简单，可靠且性能不错，但主要是同步 actor 模型。                                  |
| RQ + Redis                   | 简单易懂      | 上手快，但不太适合当前 LangChain/LLM 这种 async-heavy 代码。                          |
| arq + Redis                  | 最适合当前代码库  | 原生 asyncio 的 Redis 任务队列，现有 TaskRunnerService._dispatch_async 可以较自然复用。 |

参考资料：

- FastAPI BackgroundTasks docs: <https://fastapi.tiangolo.com/tutorial/background-tasks/>
- Celery introduction: <https://docs.celeryq.dev/en/main/getting-started/introduction.html>
- Dramatiq docs: <https://dramatiq.io/>
- arq docs: <https://arq-docs.helpmanual.io/>

## 推荐方案

第一版建议使用 **Redis + arq**。

原因：

- 当前任务执行器内部已经是 async。
- LLM、embedding、文档处理都偏 I/O 密集。
- 项目已经有 QueueService 抽象，API 侧改动可以收敛在队列边界。
- arq 提供真正的 worker 进程、Redis 持久队列、重试机制和 async 函数支持，不需要强行套 Celery 式同步包装。

只有在后续出现这些需求时，再考虑 Celery：

- 多个具名队列和复杂路由
- Flower 这类成熟运维监控工具
- 大规模周期任务
- RabbitMQ broker 支持
- 团队已有 Celery 使用经验

## 目标架构

Frontend

- > FastAPI API
  - > ChatService
    - > LangChain streaming agent 处理聊天回复
    - > tools 将生成任务入队
  - > Resource services
    - > 将后台任务入队
  - > Redis
    - > arq worker
      - > TaskRunnerService
        - > document processing
        - > FlashcardAgent
        - > QuizAgent
        - > NoteAgent
        - > MindMapAgent
        - > PostgreSQL / pgvector

API 负责：

- 校验请求
- 必要时创建占位数据库记录
- 投递任务到队列
- 快速返回 job\_id / resource\_id

Worker 负责：

- 从 Redis 消费任务
- 运行 TaskRunnerService
- 更新生成资源和文档状态
- 记录错误日志

- 必要时把 job/resource 标记为 failed

## 配置变更

新增环境变量：

```
TASK_QUEUE_BACKEND=arq
REDIS_URL=redis://localhost:6379/0
TASK_QUEUE_NAME=edu-agent:tasks
TASK_JOB_TIMEOUT_SECONDS=900
TASK_JOB_MAX_TRIES=3
```

保留本地开发 fallback：

```
TASK_QUEUE_BACKEND=local
```

这样没有安装 Redis 时，还能继续使用当前同步行为。

## 依赖变更

新增依赖：

```
src/shared/queue/pyproject.toml
```

```
dependencies = [
    "pydantic>=2.12.5",
    "arq>=0.28.0",
]
```

```
src/edu-worker/pyproject.toml
```

```
dependencies = [  
    "edu-core",  
    "edu-ai",  
    "edu-queue",  
    "pydantic-settings>=2.12.0",  
    "rich>=14.2.0",  
    "arq>=0.28.0",  
]
```

## 文件级修改计划

### 1. 保留消息 schema

继续把 `src/shared/queue/src/edu_queue/schemas.py` 作为 API 和 worker 之间的共享任务协议。

后续可以添加可选元数据：

```
class QueueTaskMessage(TypedDict):  
    type: TaskType  
    data: TaskData  
    job_id: NotRequired[str]  
    requested_by: NotRequired[str]
```

第一阶段不要改任务 payload，降低迁移风险。

### 2. 拆分队列实现

把 `src/shared/queue/src/edu_queue/service.py` 重构成：

```
src/shared/queue/src/edu_queue/service.py  
src/shared/queue/src/edu_queue/local.py  
src/shared/queue/src/edu_queue/arq_queue.py
```

建议接口：

```
class QueueServiceProtocol(Protocol):  
    async def send_message(self, message: QueueTaskMessage) -> str: ...
```

本地同步实现：

```
class LocalQueueService:
    def __init__(self, task_handler):
        self.task_handler = task_handler

    async def send_message(self, message):
        self.task_handler(message)
        return "local-sync"
```

arq 实现：

```
class ArqQueueService:
    def __init__(self, redis_pool, queue_name: str):
        self.redis = redis_pool
        self.queue_name = queue_name

    async def send_message(self, message):
        job = await self.redis.enqueue_job(
            "run_task",
            message,
            _queue_name=self.queue_name,
        )
        return job.job_id
```

### 3. 更新 FastAPI 依赖构造

修改 src/edu-api/config.py：

```
task_queue_backend: str = "local"
redis_url: str = "redis://localhost:6379/0"
task_job_timeout_seconds: int = 900
task_job_max_tries: int = 3
```

修改 src/edu-api/dependencies.py：

- 如果 TASK\_QUEUE\_BACKEND=local，返回 LocalQueueService
- 如果 TASK\_QUEUE\_BACKEND=arq，使用 Redis pool 创建 ArqQueueService

由于当前 FastAPI dependency 构造大多是同步函数，不建议每次请求里临时创建 Redis 连接。更合理的方式是在 app lifespan 中创建 Redis pool。

推荐放到 app state：

```
app.state.redis_pool = await create_pool(RedisSettings.from_dsn(settings.redis_url))
```

然后 dependency 从 `request.app.state.redis_pool` 读取连接池。

## 4. 将服务层调用改成 `await enqueue`

当前服务层调用是：

```
self.queue_service.send_message(task_message)
```

真实队列 API 应改成 `async`：

```
job_id = await self.queue_service.send_message(task_message)
```

受影响的服务包括：

- `src/shared/core/src/edu_core/services/documents.py`
- `src/shared/core/src/edu_core/services/flashcard_groups.py`
- `src/shared/core/src/edu_core/services/quizzes.py`
- `src/shared/core/src/edu_core/services/notes.py`
- `src/shared/core/src/edu_core/services/mind_maps.py`

如果第一阶段把所有服务方法都改成 `async` 影响太大，可以先提供同步兼容方法：

```
def send_message_sync(self, message):  
    return anyio.from_thread.run(self.send_message, message)
```

但更干净的目标状态是：所有“投递后台任务”的服务方法都改成 `async`。

## 5. 让 `edu-worker` 重新成为真正 worker

用 `arq worker` 入口替换当前 `src/edu-worker/main.py`。

建议布局：

```
src/edu-worker/main.py  
src/edu-worker/worker.py
```

worker 函数：

```
async def run_task(ctx, message: QueueTaskMessage):
    settings = get_settings()
    init_db(settings.database_url)
    search_service = SearchService(...)
    runner = TaskRunnerService(...)
    await runner._dispatch_async(message)
```

worker 配置:

```
class WorkerSettings:
    functions = [run_task]
    redis_settings = RedisSettings.from_dsn(settings.redis_url)
    queue_name = settings.task_queue_name
    job_timeout = settings.task_job_timeout_seconds
    max_tries = settings.task_job_max_tries
```

启动命令:

```
uv run --package edu-worker arq worker.WorkerSettings
```

最终 module path 要以实际文件结构为准。

## 6. 增加 job/resource 状态跟踪

前端需要知道后台任务当前是 pending、running、done 还是 failed。

文档处理已经有 DocumentStatus，需要确认至少覆盖:

```
uploaded -> processing -> indexed
uploaded -> processing -> failed
```

生成类资源如果还没有状态字段，需要补:

```
pending
running
completed
failed
```

最小可行方案:



- 入队前创建占位记录
- 状态设为 `pending`
- worker 开始处理时设为 `running`
- worker 写入 DB 成功后设为 `completed`
- worker 捕获异常后设为 `failed`，并记录错误信息

## 7. 调整 API 响应

生成类接口应快速返回：

```
{
  "status": "queued",
  "job_id": "...",
  "resource_id": "..."
}
```

如果缺少查询接口，需要补轮询接口：

```
GET /api/v1/documents/{document_id}
GET /api/v1/flashcard-groups/{group_id}
GET /api/v1/quizzes/{quiz_id}
GET /api/v1/notes/{note_id}
GET /api/v1/mind-maps/{mind_map_id}
```

后续可选：

```
GET /api/v1/jobs/{job_id}
```

arq 支持 job result，但对这个项目来说，数据库更应该作为资源状态的事实来源，因为生成结果本来就存储在业务表里。

## 最小实施顺序

### Phase 1: 基础设施

1. 增加 Redis/arq 配置。
2. 增加 arq 依赖。
3. 新增 `ArqQueueService`。

4. 保留当前 `LocalQueueService`。
5. 新增调用 `TaskRunnerService._dispatch_async` 的 worker 函数。

## Phase 2: API 入队

1. 更新 API dependency wiring。
2. 让任务入队返回 `job_id`。
3. 尽量保持 endpoint response 兼容。
4. 增加 `TASK_QUEUE_BACKEND=local fallback`。

## Phase 3: 状态与可靠性

1. 增加或统一状态字段。
2. 失败任务写回 DB。
3. 增加 retry 配置。
4. 增加结构化日志。

## Phase 4: 前端体验

1. 资源生成开始后展示 `queued/running` 状态。
2. 轮询资源状态。
3. 展示 `failed` 状态和 `retry` 操作。

## 迁移后的本地启动命令

先用你选择的方式启动 PostgreSQL 和 Redis。

后端：

```
cd D:\github\Soft-cup
uv run --package edu-api python src/edu-api/main.py
```

Worker：

```
cd D:\github\Soft-cup
uv run --package edu-worker arq worker.WorkerSettings
```

前端：

```
cd D:\github\Soft-cup\src\edu-web
npm run dev -- --host 127.0.0.1
```

## 风险

- 当前 service 层大多是同步代码，把入队路径改成 async 可能会影响多个 router 和前端预期。
- PostgreSQL session 生命周期必须适配 worker。每个 worker job 应该创建并关闭自己的 DB session。
- LLM 任务需要具备足够的幂等性。重试发生在部分写入之后时，必须避免重复创建 flashcards、quiz questions、notes、mind maps。
- 当 TASK\_QUEUE\_BACKEND=arq 时，Redis 会成为必需依赖。
- 如果 Windows 本地不用 Docker，Redis 安装会成为新的本地前置条件。

## 验收标准

- API 在启动文档处理或资源生成后能快速返回。
- worker 日志能看到独立进程接收并执行 job。
- 停止 API 后，Redis 中未执行的任务不会丢失。
- 失败 job 会把对应 document/resource 标记为 failed。
- 重试 job 不会生成重复 flashcards、quiz questions、notes 或 mind maps。
- TASK\_QUEUE\_BACKEND=local 仍然可用于简单本地开发。

## 最终建议

优先实现 **arq + Redis**，因为它最贴合当前已有的 async TaskRunnerService，迁移成本比 Celery 更小。

如果项目后续需要更企业化的任务系统、复杂路由、成熟监控和团队通用经验，可以再把同一层队列边界迁移到 Celery。真正重要的设计点不是具体选哪个库，而是把 API 请求处理和后台任务执行通过一个持久队列契约分离开。